

Cross-Platform Rust Loader Delivers Modified Sliver Implant via Cloud Storage Abuse

Will Seligman · RAID Team (Research, Adversary Intelligence, and Detections) · July 6, 2026

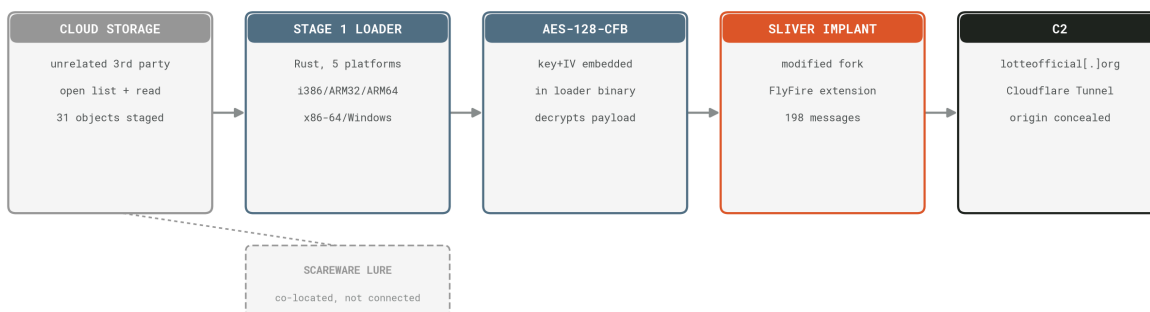
We identified a two-stage malware operation: a Rust-compiled downloader that fetches and AES-128-CFB decrypts a second payload confirmed to be a modified build of Sliver, the open-source C2 framework. Both stages are staged inside a misconfigured cloud storage bucket belonging to an unrelated third party, and command-and-control runs behind Cloudflare Tunnel, concealing the operator's true origin server. RADICL tracks this activity as **UNC-RAID-M8280**.

KEY FINDINGS

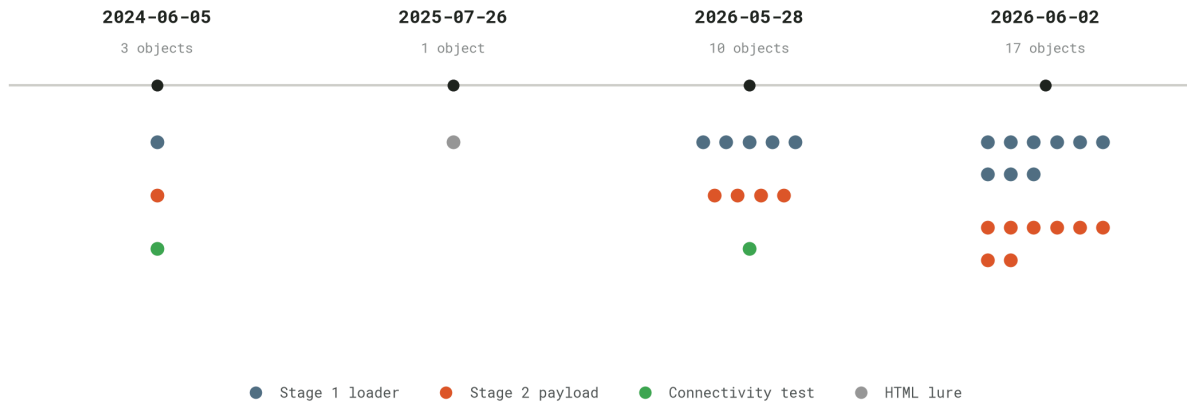
- A Rust-compiled loader downloads and AES-128-CFB decrypts a second-stage payload confirmed to be a modified build of Sliver, the open-source C2 framework.
- One build pipeline covers five platforms — i386, ARM32, ARM64, and x86-64 Linux, plus Windows — with per-build AES keys patched into an otherwise-identical compiled template.
- The Sliver payload carries a custom protobuf extension (FlyFireReq, FlyFireUpgradeReq, FlyFireToolsUploadReq) absent from the public Sliver source tree.
- Both stages are staged inside a misconfigured cloud storage bucket belonging to an unrelated third party.
- Command-and-control sits behind Cloudflare Tunnel, concealing the operator's true origin server. All observed C2 endpoints returned a tunnel-not-connected error at analysis time.
- 31 unique objects recovered across four upload waves, June 2024 through June 2026.

Infection chain

Rust loader decrypts a modified Sliver implant staged in a misconfigured cloud storage bucket



Upload activity



1. LOADER

All loader samples are Rust binaries statically linking `reqwest/hyper/tokio` for HTTP, `aes/cfb-mode` for AES-128-CFB, and `rand_chacha`. Every build shares one binary template: a 32-byte block holding a 16-byte key followed by a 16-byte IV, immediately followed by a drop-path string. A patcher tool stamps new key/IV/path values into this block per build; same-wave builds across every architecture are byte-identical outside that one patched region.

Summary

Arch	Samples	Format	Linking	Entry point	Symbols	Size
i386	3 (2 generations)	ELF32 EXEC	static	0x804be61 / 0x804bad5	stripped	1,381,524 / 1,444,352 B
ARM32 (EABI5)	3	ELF32 EXEC	static	0x13f54	not stripped	1,832,272 B
ARM64	4	ELF64 EXEC	static	0x4031bc	stripped	2,016,000 B
x86-64	4	ELF64 DYN, static-PIE	static, no NEEDED/INTERP	0x14c0a	stripped	1,569,392 B
Windows	3	PE32+ GUI, MinGW-w64	static	n/a	stripped, no PDB	1,380,352 B

i386 — two build generations

L-I386-1 (1,381,524 bytes, entry `0x804be61`) predates the rest of the set by roughly two years. L-I386-2 and L-I386-3 (1,444,352 bytes, entry `0x804bad5`) are from the 2026 waves and are byte-identical to each other outside the key/IV/path block. Different size and different entry point across the gap — a rebuild, not a re-key.

```
$ readelf -h L-I386-1 L-I386-2 | grep -E 'Type:|Entry point'
```

```
L-I386-1 (2024-06-05 wave, 1,381,524 bytes)
  Entry point address: 0x804be61
L-I386-2 (2026-06-02 wave, 1,444,352 bytes)
  Entry point address: 0x804bad5
```

All three are stripped. Key/IV recovered from the 32 bytes preceding each binary's embedded /tmp/ path:

	Key	IV
L-I386-1	d0e2264dbad6c9a087a4c3bada926317	108e7e7b429f1d1eb188134a8af9601c
L-I386-2	5a912cc38d31d60b8578ae3368076fed	dcedd5cbc3e07750f62cb79cec8e5252
L-I386-3	b5f233ba80dc1a3887d382a2ec929860	f242a50931193e5989c90eb833a60146

ARM32 – unstripped, confirms the code path directly

All three ARM32 samples share one build (entry 0x13f54, 1,832,272 bytes) and, unlike every other architecture in this set, ship with symbols intact:

```
$ nm L-ARM32-1 | c++filt | grep -E \
'AES_KEY|AES_IV|TMP_PATH|aes128_encrypt::h|encrypt_block::h|decrypt_shellcode_stub|\
linux::main::h|linux::download.*closure|can_access_internet' | sed -E 's/::h[0-9a-f]+$//'
```

```
000de2d0 T AES_IV
000de2c0 T AES_KEY
000de2e0 T TMP_PATH
0001eac4 t aes::soft::fixslice::aes128_encrypt
00014b4c t core::ptr::drop_in_place<linux::download::{closure}>
00014d28 t core::ptr::drop_in_place<linux::can_access_internet::{closure}>
00015ad0 t linux::decrypt_shellcode_stub
00018b18 t linux::main
00016390 t linux::download::{closure}
000156a0 t <aes::soft::Aes128 as cipher::block::BlockEncrypt>::encrypt_block
```

The key, IV, and drop-path string sit contiguous in memory, in that order:

```
$ objdump -s -j .text L-ARM32-1
```

```
Contents of section .text:
... 84c8fceb 7cf60700 7c46f4ff 5cf60700 ....|...|F..\...
[KEY] 1a57aec9 eba263c6 801fa591 978ce36d .W....c.....m
[IV] 7da6111e 87ac5cd7 25f75c5c a2ffbdb9 }.....\.%.\....
[PATH] 2f746d70 2f363163 65306138 30623830 /tmp/61ce0a80b8
... 64666230 31316663 65656431 37383262 dfb011fceed1782b
```

The source crate is named `linux`; control flow is `can_access_internet` → `download` → `decrypt_shellcode_stub`. Only an `encrypt` primitive exists in the binary — CFB decryption reuses the block cipher's `encrypt` operation on the keystream, so no separate `decrypt` routine is ever compiled in. The `aes` crate's `soft::fixslice` implementation is a bitsliced, table-free AES — it doesn't use the classic S-box lookup table, which is why that byte sequence doesn't appear anywhere in these binaries.

ARM64

Four samples, one build (entry `0x4031bc`, 2,016,000 bytes). Symbols aren't retained here despite `file` reporting "not stripped" — key/IV extraction used the same drop-path-marker method as the stripped architectures.

x86-64

Four samples, one build (entry `0x14c0a`, 1,569,392 bytes). `file` calls these "dynamically linked" — misleading. No `NEEDED` entries, no `PT_INTERP` segment: this is static-PIE, fully self-contained.



```
$ readelf -d L-X64-1 | grep -E 'NEEDED|FLAGS_1'

0x000000006ffffffb (FLAGS_1)          Flags: NOW PIE

$ readelf -l L-X64-1 | grep INTERP
# (no output -- grep found no PT_INTERP segment; no NEEDED entries either)
```

Windows — MinGW-w64

Three samples, 1,380,352 bytes, byte-identical outside a 270-byte patched window (offset `0xd2131`–`0xd223e`) holding the same key/IV/path template, adapted to `c:/users/public/downloads/<hex>`.

Toolchain is GNU/MinGW-w64, not MSVC: `MajorLinkerVersion=2.34` is a 1d version number, no Rich/DanS marker is present anywhere in the file, and the import table pulls `msvcrt.dll` with legacy CRT startup symbols (`__getmainargs`, `__initenv`, `_acmdln`) characteristic of MinGW's `crt1.o` rather than MSVC's UCRT startup. Build timestamp `1751339644` (2025-07-01T03:14:04Z) is identical across all three.

192 imported functions across 14 DLLs. No `CryptoAPI` imports — AES-CFB is entirely the statically-linked Rust crates, same as the Linux builds. `OpenProcess`, `VirtualAllocEx`, `WriteProcessMemory`, `ResumeThread`, and `CreateProcessA/W` are all present, but `NtUnmapViewOfSection` and `SetThreadContext/GetThreadContext` are absent, as is `CreateRemoteThread` — the import table alone supports a drop-to-disk-and-execute loader, not confirmed process hollowing.

Cross-architecture notes

- Key reuse spans architectures within a wave, not just within one architecture — the same AES key patches into the i386 build and the Windows build in the same batch.

- No persistence, no mutex, no anti-debug or anti-VM strings anywhere in the loader, any architecture. Any persistence on a compromised host comes from the operator via the second stage, not the loader itself.

2. PAYLOAD

Decryption

All 13 second-stage payloads are AES-128-CFB ciphertext — no header, no magic bytes, uniform ~8.0 bits/byte entropy end to end. All 13 keys trace directly back to loaders in this set. Only nine distinct keys cover the 17 loaders, and the reuse isn't confined to one architecture — the same key can span multiple loader architectures and decrypt multiple payloads in different formats:

Key	Loaders sharing it	Payloads it decrypts
1	L-I386-1	P-ARM64-3
2	L-WIN-1, L-I386-3	P-I386-1, P-X64-1, P-X64-SC-3
3	L-WIN-3, L-I386-2, L-X64-4, L-ARM64-4	P-ARM32-2
4	L-WIN-2, L-X64-2	P-X64-SC-1
5	L-X64-1, L-ARM32-1	P-X64-SC-2
6	L-X64-3, L-ARM32-2	P-ARM32-1, P-ARM64-2
7	L-ARM32-3, L-ARM64-1	P-I386-2, P-X64-2
8	L-ARM64-2	P-I386-3, P-ARM64-1
9	L-ARM64-3	none recovered

Key 2 decrypts three payloads across three different formats — native i386, native x86-64, and x86-64 shellcode. Keys 6, 7, and 8 each decrypt two. The operator is minting one key per delivery batch, not per artifact — architecture and payload format don't factor into key assignment. Key 9's loader, L-ARM64-3, has no matching payload among the 13 recovered, meaning whatever it was paired with was never uploaded or already removed.

Identification

10 of 13 payloads decrypt to native ELF binaries matched to their loader's architecture. The remaining 3 decrypt to raw x86-64 shellcode with no file header — built for reflective/in-memory loading rather than execution from disk. Every payload, ELF or shellcode, carries the identical 198-message `sliverpb.*` capability surface — the shellcode packaging isn't a reduced build.

```
$ file P-ARM32-1
P-ARM32-1: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked, stripped

$ hexdump -C P-ARM32-1 | head -2
00000000  7f 45 4c 46 01 01 01 00  00 00 00 00 00 00 00 00  |.ELF.....|
00000010  02 00 28 00 01 00 00 00  04 36 08 00 34 00 00 00  |..(.....6..4...|
```

Build fingerprint across all 13: Go 1.19.3, CGO_ENABLED=0, -trimpath=true (no leaked developer paths), module path github.com/bishopfox/sliver, version (devel).

```
$ strings P-ARM32-1 | grep -E \
'^go1\.\d\.\d$|^path[:space:]+|^mod[:space:]+|^build[:space:]+(-trimpath|CGO_ENABLED)'

go1.19.3
build    -trimpath=true
build    CGO_ENABLED=0
path     github.com/bishopfox/sliver
mod      github.com/bishopfox/sliver (devel)
```

A modified fork

All 13 payloads carry protobuf message types compiled into the standard

github.com/bishopfox/sliver/protobuf/sliverpb package that do not exist in the public Sliver source:

FlyFireReq/FlyFireResp, FlyFireUpgradeReq (versioned self-upgrade, carries a file name and payload), and

FlyFireToolsUploadReq/FlyFireToolsUploadResp (per-architecture tool upload with an MD5 checksum). Full getter methods are compiled in for every field — a genuine schema extension, not a stray string.

```
$ strings P-X64-1 | grep -E \
'\.\.(\*FlyFire(Req|UpgradeReq|ToolsUploadReq))\.\.Get'

github.com/bishopfox/sliver/protobuf/sliverpb.(*FlyFireReq).GetRequest
github.com/bishopfox/sliver/protobuf/sliverpb.(*FlyFireUpgradeReq).GetVersion
github.com/bishopfox/sliver/protobuf/sliverpb.(*FlyFireUpgradeReq).GetFileName
github.com/bishopfox/sliver/protobuf/sliverpb.(*FlyFireUpgradeReq).GetFileData
github.com/bishopfox/sliver/protobuf/sliverpb.(*FlyFireUpgradeReq).GetRequest
github.com/bishopfox/sliver/protobuf/sliverpb.(*FlyFireToolsUploadReq).GetTaskID
github.com/bishopfox/sliver/protobuf/sliverpb.(*FlyFireToolsUploadReq).GetFileData
github.com/bishopfox/sliver/protobuf/sliverpb.(*FlyFireToolsUploadReq).GetFileArch
github.com/bishopfox/sliver/protobuf/sliverpb.(*FlyFireToolsUploadReq).GetFileName
github.com/bishopfox/sliver/protobuf/sliverpb.(*FlyFireToolsUploadReq).GetMd5Sum
github.com/bishopfox/sliver/protobuf/sliverpb.(*FlyFireToolsUploadReq).GetRequest
```

Capability surface

File system: Ls, Mkdir, Mv, Rm, Cd, Pwd, Download, Upload, FileInfo

Execution: Execute, ExecuteAssembly, InvokeExecuteAssemblyReq, InvokeInProcExecuteAssemblyReq, SpawnDll, Sideload, Task, Ps, ProcessDump, LinuxMemFdExecuteReq (fileless exec via memfd_create), WindowsMemExecuteReq

Privilege escalation: GetSystem, GetPrivs, Impersonate, MakeToken, RevToSelf, RunAs, CurrentTokenOwner, WindowsPrivilegeEntry

Persistence: Backdoor, ServiceInfo, StartServiceReq, StopServiceReq, Migrate

Windows registry: RegistryCreateKey, RegistryDeleteKey, RegistryRead, RegistryWrite, RegistrySubKeyList, RegistryValuesList

Networking and pivoting: Portfwd, RPortfwd, Socks, StartImplantLocalSocks5Req, WGSocks/WGSocksServer (WireGuard SOCKS), WGPortForward, WGTCPForwarder, NetConnPivot, Netstat, Ifconfig, Tunnel/TunnelData

Multi-hop pivoting: PivotHello, PivotListener, PivotPeer, PivotServerKeyExchange, PivotStartListenerReq

C2 session: BeaconRegister, BeaconTasks, DNSBlockHeader, DNSSessionInit, HTTPSessionInit, Envelope, OpenSession, Reconfigure, PollInterval, KillReq, WebhookTunnelData

Reconnaissance: Screenshot, EnvInfo, camera and interface enumeration via the vendored library set below

Extension framework: CallExtension, ListExtensions, RegisterExtension

Interactive access: Shell, SSHCommand

Fork additions: FlyFireReq/Resp, FlyFireUpgradeReq, FlyFireToolsUploadReq/Resp

Vendored libraries

Library	Capability
armon/go-socks5	SOCKS5 proxy backing the pivoting commands
hashicorp/yamux	Session/tunnel stream multiplexing
fatedier/kcp-go	KCP (reliable UDP) transport
Azure/go-ntlmssp	NTLM authentication through corporate proxies
BurntSushi/xgb	X11 screenshot/screen capture on Linux
gofrs/uuid	Session/task/beacon identifiers
pion/stun, pion/dtls	NAT traversal / pivoting transport

User-Agent strings

13 unique UAs, one per binary, drawn from Sliver's default randomized pool — 11 Windows-shaped, 2 Linux-shaped, with synthetic Chrome version numbers that don't match real releases. Not tied to the binary's actual target platform.

C2 domains

Domain	Payloads
sim[.]lotteofficial[.]org	5 payloads (i386, x86-64 native, x86-64 shellcode x2)
check[.]lotteofficial[.]org	3 payloads (ARM32, ARM64, i386)
demo[.]lotteofficial[.]org	2 payloads (i386, ARM32)
update[.]lotteofficial[.]org	2 payloads (ARM64, x86-64 shellcode)
update[.]traceroute[.]vip (https)	1 payload (ARM64, 2024 wave)

3. INFRASTRUCTURE

Both stages are staged inside a misconfigured cloud storage bucket — an unrelated company's storage, not attacker-owned infrastructure, with anonymous list and read access enabled. The malware objects sit alongside years of that company's own operational data, giving the operation free, high-reputation hosting with no infrastructure footprint of its own. An unrelated Chinese-language scareware page (a Ministry of Public Security impersonation, hash LURE-1) was also present in the same bucket, with no functional connection to the loader/implant chain.

Command-and-control runs on a domain registered through NameCheap in December 2025, Cloudflare-fronted with certificates from Google Trust Services, plus one earlier domain (traceroute[.]vip) tied to the 2024 wave that had since lapsed into domain parking and finally NXDOMAIN. Both sit behind Cloudflare Tunnel — the operator's real origin server is never exposed via DNS or direct connection.

At the time of this analysis, every C2 subdomain returns Cloudflare error 1033 (tunnel not connected), reproduced through different methods.

Two additional subdomains on the same apex domain — api[.]lotteofficial[.]org and music[.]lotteofficial[.]org — were not observed in any recovered payload but return the identical Cloudflare Tunnel error, indicating shared operator infrastructure with no confirmed sample tying them to this campaign.

```
$ curl -sS -I https://sim.lotteofficial.org/

HTTP/1.1 530 <none>
Content-Type: text/plain; charset=UTF-8
Content-Length: 17
Server: cloudflare

$ curl -sS https://sim.lotteofficial.org/
error code: 1033
```

One subdomain has a public historical scan (December 2025, status 200) serving a verbatim copy of the real `systemd sysupdate.d` manual page — cover content for anyone browsing the domain directly without the request signature an implant would send. Live reproduction returns the same tunnel error as everything else; the capture reflects a period when the tunnel was still connected.

Remediation

RADICL reported the abused storage bucket to the hosting cloud provider. The provider confirmed the reported content had been mitigated, removing the operator's staging and delivery point for both loader and payload stages.

4. DETECTION

Loader: a process writing a file to `/tmp/<64-100 hex chars>` (Linux) or `c:\users\public\downloads\<64-100 hex chars>` (Windows) and executing it shortly after is the strongest host indicator. Rust build-artifact strings (`aes-0.7.5`, `cfb-mode-0.7.1`, `decrypt_shellcode_stub`) survive stripping and are stable across build generations.

Payload: the `FlyFire*` protobuf strings are a low-collision signature for this specific fork if the payload is ever recovered unencrypted. `memfd_create`-based fileless execution (Linux) and unbacked executable memory regions (Windows) match the implant's compiled capabilities. Domain-based blocking of the C2 domains is more effective than IP-based blocking, given Cloudflare Tunnel's shared, rotating edge addresses.

Infrastructure: this delivery chain depends entirely on a cloud storage bucket with open list/read permissions.

INDICATORS OF COMPROMISE

ID	SHA-256	Size	Role
L-WIN-1	15129930ccc8ce4d51fe4d4a3a870cc5d236e51dd0d631b8f7034d892091811d	1,380,352	Loader (Windows)
L-WIN-2	45e43d5dab0436a84a15fea3a87fd6d4356c0e6e1b9093118551877d100bec18	1,380,352	Loader (Windows)
L-WIN-3	e752a754500947fe040cd15118e25e0a47fb58f98c25df716415be6f218159ac	1,380,352	Loader (Windows)
L-I386-1	ec188c14a8498774610961557ad1e911b0b6471eb144d0dab22fc4fedb66aa06	1,381,524	Loader (i386)
L-I386-3	e21448d9cf4baf67036aa42569add5e1d61ed99187b84fe38b41d6229ca7bf58	1,444,352	Loader (i386)
L-I386-2	a87e19e5a069d37d440f929c7598df4f610e82f741aec448d88c5dcfbae610c0	1,444,352	Loader (i386)
L-X64-1	8146f23ceb065296376da9b43cb2f4e1744ccf7dc498482f30601bbc1ebc92fe	1,569,392	Loader (x86-64)
L-X64-2	0ba7c5966efb4babb60f74cd4c6905c0f5396ad681933d5f7eec9476e520d4e3	1,569,392	Loader (x86-64)
L-X64-4	e470aac5a5a2dc9c85d43d5b4c91eaa0c7ee17ea069c4f9d64bccfd084a0eba7	1,569,392	Loader (x86-64)
L-X64-3	89d6582190f2e3e1974081c7b724145316c4ed096977667f5d9c22dab687bee1	1,569,392	Loader (x86-64)
L-ARM32-1	68cc6d10e471684dc12854dbcde358a79ba75196a8a8483e18c95a38b7570799	1,832,272	Loader (ARM32)
L-ARM32-2	93144e0ad2c9d7be4835b186c9c66249315e4eea578208db09d538dc9c3bfd36	1,832,272	Loader (ARM32)
L-ARM32-3	0ff5dcc2e16fd793a5be3608d7b8b42101496ee017c381975a6ed05bb326dc49	1,832,272	Loader (ARM32)
L-ARM64-1	6f1f689f7142a20aeee707234f177733cc49030a3be3f7a7b6aa3da3c20e1f6e	2,016,000	Loader (ARM64)
L-ARM64-2	0f4e170f68f756a478885aae2b1bbb98a79cbac21fcb9d5c0a6d90aac1940877	2,016,000	Loader (ARM64)
L-ARM64-3	200ac78f7a62ca33ad81a00e56106046363a8c2d702ff0248a01c1d2c0d29d8c	2,016,000	Loader (ARM64)
L-ARM64-4	094ab5c4eca64bbe80a1fb242de9f49a4f9246b029754d0d9049fe6bcf8ad280	2,016,000	Loader (ARM64)

ID	SHA-256	Size	Role
P-ARM32-1	e93e236ec56f2eff761bd7ef5ebc073e107905cf10978bdb106688e2a7f83158	10,682,368	Payload – ARM32
P-ARM32-2	1a2beac30028a2368f7af0893e86a015cebce9d5644bb787b18fce10f4b025c4	10,682,368	Payload – ARM32
P-I386-1	91c584b2897a59960f22bcb2391a25ab5a4fd498a76f08c7a3c8428922eed929	10,694,656	Payload – i386
P-I386-2	dadad7a7524979a5688287aa6e1fa0e0476606db5e1a4148a8cfff07e05831bf	10,694,656	Payload – i386
P-I386-3	29a8c39617ef25bffe7925a11d08ed550ace3dde3df571388c78b12662a7e94	10,694,656	Payload – i386
P-ARM64-1	de78c0bd5a1cad3c69aca691e29ae10a31d83bb95cf8dd06b47dd733222c6519	10,747,904	Payload – ARM64
P-ARM64-2	7940d87bb7e413a3f0a20c90a86f973ca7174e04c3c7d1b71c9d5c7043dcb8ea	10,747,904	Payload – ARM64
P-ARM64-3	f507437b764a73e819641443a946e0b252e0d46ac029189615f67b348c97401f	10,747,904	Payload – ARM64
P-X64-1	56c7dbafefafb3d7b8927aab4bdcea528ab5380d59e51b3ca28a7301d13eb445	11,493,376	Payload – x86-64
P-X64-2	76d2e41a6d0efc2eb895d61477ff60f957722d5af42265957e54a70feaf00678	11,493,376	Payload – x86-64
P-X64-SC-1	5b734ddfab4e87d0805f5d2b76a2ba0b9da3e413837539da9c53d33cc6dc018c	11,746,524	Payload – x86-64 shellcode
P-X64-SC-2	5de704b2c88a2c56ab9db3b02d029ff52c7b1cb16e3190aa3fc7d9103a6e2522	11,746,524	Payload – x86-64 shellcode
P-X64-SC-3	055ba3a7328613e5a162a6e38f0758fcd7faae06786f4e8d955d2be1432ff8fb	11,747,036	Payload – x86-64 shellcode
LURE-1	a3ec88293cfbd0be4c8331a4397e12f8a97067073bec94d6666eb69519d9ea6e	4,624	Scareware HTML lure

Network

Indicator	Type
sim[.]lotteofficial[.]org	C2
check[.]lotteofficial[.]org	C2
demo[.]lotteofficial[.]org	C2
update[.]lotteofficial[.]org	C2
update[.]traceroute[.]vip	C2

Indicator	Type
api[.]lotteofficial[.]org	C2 (no payload observed)
music[.]lotteofficial[.]org	C2 (no payload observed)

File system

Pattern	OS
/tmp/[0-9a-f]{64,100}	Linux drop path
c:\users\public\downloads\[0-9a-f]{64,100}	Windows drop path

YARA

```
rule Rust_Stager_AES_CFB_Sliver_Loader {
  meta:
    description = "Rust-compiled multi-platform stager that downloads and AES-CFB decrypts a Sliver-family implant"
    date = "2026-07-09"
  strings:
    $sym1 = "linux::download" ascii
    $sym2 = "decrypt_shellcode_stub" ascii
    $sym3 = "can_access_internet" ascii
    $crate_aes = "aes-0.7.5" ascii
    $crate_cfb = "cfb-mode-0.7.1" ascii
    $droppath_lin = /\tmp\[0-9a-f]{60,100}/ ascii
    $droppath_win = "c:/users/public/downloads/" ascii nocase
  condition:
    (2 of ($sym*)) or (all of ($crate_aes, $crate_cfb)) or $droppath_lin or $droppath_win
}
```

```
rule Sliver_Implant_Custom_FlyFire_Extension {
  meta:
    description = "Sliver C2 implant carrying non-stock FlyFire* protobuf extension"
    date = "2026-07-09"
  strings:
    $stock1 = "github.com/bishopfox/sliver" ascii
    $stock2 = "sliverpb." ascii
    $custom1 = "FlyFireReq" ascii
    $custom2 = "FlyFireUpgradeReq" ascii
    $custom3 = "FlyFireToolsUploadReq" ascii
  condition:
    (1 of ($stock*)) and (1 of ($custom*))
}
```